🧩 Problemas

Tabla de contenidos

1 Arreglos manuales completos 1
2 Lista enlazada 1
3 Pila como lista enlazada 2
4 Cola como lista doblemente enlazada 2
5 Eliminar nodo de lista enlazada 2
6 Arreglos indexados 2
7 Pilas y colas ruidosas 3
8 Pilas y colas limitadas 3
9 Inversión recursiva 3

1 Arreglos manuales completos

El recurso Arreglos manuales muestra como implementar un arreglo en Python sin utilizar ninguna estructura de datos de alto nivel como las listas o las tuplas.

Si bien la clase `Array` provee métodos elementales para interactuar con los arreglos, carece de otros métodos convenientes que encontramos en estructuras como las listas.

Implemente los siguientes métodos en la clase `Array`.

- `append(self, valor)`: agrega el elemento `valor` al final del arreglo. Análogo al método `append` de las listas.
- `extend(self, arreglo)`: agrega todos los elementos del arreglo `arreglo` al final del arreglo. Análogo al método `extend` de las listas.
- `reverse(self)`: invierte el orden de los elementos en el arreglo, modificandolo *in-place*.
- `count(self, valor)`: devuelve la cantidad de veces que aparece `valor` en el arreglo.
- `clear(self)`: elimina todos los elementos del arreglo, dejandolo vacío *in-place*.
- `__repr__(self)`: modifíquelo para que muestre solo los tres elementos iniciales y finales cuando la cantidad de elementos sea mayor a 10. En el medio, debe mostrar `...`. Por ejemplo `Array(10, 22, -10, ..., 5, 5, 5)`.
- `__iter__(self)`: permite iterar sobre el arreglo. Debe contener un bucle *for* que entrega (con *yield*) los valores del arreglo de a uno.

Finalmente, modifique la clase `Array` para que encoja el tamaño del bloque de memoria cuando la cantidad de elementos sea menor al 20% de la capacidad.

2 Lista enlazada

Modifique la `ListaEnlazada` del apunte para que:

- Implemente el método especial `__len__`.

- Mantenga un conteo interno de la cantidad de elementos en la lista de forma tal que no sea necesario recorrerla para obtener su longitud.
- Implemente un método `eliminar_siguiente` que reciba como argumento un nodo de una lista enlazada y elimine el nodo que se encuentra inmediatamente después del nodo dado.
- Implemente un método `elimiar_valor` que reciba como argumento un valor y elimine de la lista enlazada todos los nodos que contengan ese valor.

Luego, implemente el método `reverse(self)` que invierte los elementos de la lista enlazada.

3 Pila como lista enlazada

Implemente una clase `Pila` que utilice una lista enlazada para almacenar los elementos de la pila.

4 Cola como lista doblemente enlazada

Implemente una clase `Cola` que utilice una lista doblemente enlazada para almacenar los elementos de la pila.

Implemente un método `concatenar(self, C2)` que agregue todos los elementos de la cola `C2` al final de la cola `C1` (es decir, la cola sobre la que se realiza la llamada). La operación debe ejecutarse en tiempo constante (O(1)) y debe dejar a `C2` convertida en una cola vacía.

5 Eliminar nodo de lista enlazada

Suponga que tiene acceso a un nodo ubicado en algún punto intermedio de una lista enlazada clásica, pero no tiene acceso a la lista completa. Es decir, se tiene una variable que apunta a una instancia de `Nodo`, pero no al objeto `ListaEnlazada`.

En esta situación es posible acceder a todos los valores desde este nodo hasta el final de la lista, pero no hay forma de acceder a los nodos que lo preceden.

Escriba un programa que elimine el nodo de la lista original. La lista original restante debe permanecer completa, con solo este nodo eliminado.

6 Arreglos indexados

Agregue el método `__getitem__(self, indice)` a la clase `Array` para que sea posible acceder a sus valores de la siguiente manera:

```
arreglo = Array(10, 20, 150)
arreglo[0] # 10
arreglo[1] # 20
arreglo[2] # 150
arreglo[3] # IndexError
```

Asegúrese de que el método `__getitem__` soporta índices negativos.

7 Pilas y colas ruidosas

La implementación actual de `Pila` y `Cola` devuelve `None` cuando se intenta extraer un elemento de una pila (o cola) vacía. Modifique estas clases para que eleven un error llamado `EmptyStructure`.

8 Pilas y colas limitadas

Modifique la implementación de `Pila` y `Cola` para que tengan una capacidad limitada, establecida mediante el parámetro `capacidad_maxima` el método inicializador, cuyo valor por defecto es `None`. Cuando se intente agregar un elemento en una pila o cola que está a máxia capaciad, se debe arrojar un error `FullCapacity`.

9 Inversión recursiva

Describa un algoritmo recursivo eficiente para invertir una lista simplemente enlazada.